

JobTrack

Job Application & Recruitment Management System

Complete Technical Architecture & Development Documentation

Project Name:	JobTrack System
Application Type:	Java Full Stack Enterprise Web Application
Technology Stack:	React 18 + Spring Boot 3 (Java 21) + MySQL
Author / Engineer:	Chandra Shekhar
Document Version:	1.0
Date:	September 20, 2026
Live Frontend:	https://jobtrack-frontend-inky.vercel.app
Live Backend API:	https://jobtrack-backend-xuvm.onrender.com/
GitHub Repository:	https://github.com/devchandra-web/jobtrack

Complete Technical & Development Documentation

Author: Chandra Shekhar

Version: 1.0

Date: September 20, 2026

Technology Stack: React 18 + Spring Boot 3 (Java 21) + MySQL (Aiven Cloud) + Spring Security (JWT)

Live Deployment:

- **Frontend (Vercel):** <https://jobtrack-frontend-inky.vercel.app>
- **Backend (Render):** <https://jobtrack-backend-xuvm.onrender.com/>
- **GitHub Repository:** <https://github.com/devchandra-web/jobtrack>

TABLE OF CONTENTS

1. Executive Summary & Project Overview
2. Problem Statement
3. Project Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. Technology Stack
7. System Architecture
8. Frontend Architecture
9. React Project Structure
10. Backend Architecture
11. Spring Boot Project Structure
12. Controller Documentation
13. Service Layer Documentation
14. Repository Layer Documentation
15. Entity & JPA Documentation
16. Database Architecture
17. Entity-Relationship (ER) Diagram
18. Authentication Architecture
19. Authorization & Role-Based Access Control (RBAC)
20. API Specification & Documentation
21. API Request/Response Lifecycle
22. Frontend-Backend Integration
23. Feature & Module Documentation
24. Complete User Flows

- 25. Development Workflow
- 26. Git & GitHub Workflow
- 27. Environment Configuration
- 28. Local Development Setup Guide
- 29. Testing & Quality Assurance
- 30. Error Handling Strategy
- 31. Security Implementation
- 32. Performance Optimization
- 33. Responsive Design Implementation
- 34. Deployment Architecture
- 35. Continuous Integration / Continuous Deployment (CI/CD)
- 36. Troubleshooting Guide
- 37. Code Quality & Architectural Observations
- 38. Future Enhancements & Roadmap
- 39. Developer Onboarding Guide
- 40. Developer Quick Reference
- 41. Final Project Summary

1. PROJECT OVERVIEW

Executive Summary

JobTrack is a enterprise-grade, full-stack recruitment and job application management platform designed to connect job seekers (Candidates), hiring managers (Recruiters), and system managers (Admins). Built using **React 18** on the frontend and **Spring Boot 3 (Java 21)** on the backend, JobTrack leverages **MySQL** hosted on Aiven Cloud for relational data storage and is fully deployed across **Vercel** and **Render** cloud infrastructures.

The system provides a seamless, secure, role-based workflow allowing candidates to search for jobs and track application status, recruiters to post listings and manage applicants through hiring pipelines, and administrators to oversee user accounts and system metrics.

Key Target Users

1. **Job Candidates:** Individuals searching for career opportunities, submitting applications with cover letters and resumes, and tracking real-time status updates (APPLIED, REVIEWING, SHORTLISTED, REJECTED, ACCEPTED).
2. **Employers / Recruiters:** Organizations posting job vacancies, managing job descriptions, reviewing candidate applications, and updating hiring pipeline stages.
3. **System Administrators:** System managers responsible for user management, role governance, system health monitoring, and analytics.

2. PROBLEM STATEMENT

Existing Problem

Traditional job application processes rely on fragmented communication channels—emails, spreadsheets, and manual tracking—leading to lost candidate data, delayed hiring feedback, lack of transparency for job seekers, and operational overhead for recruiters.

Solution Provided by JobTrack

JobTrack solves these challenges by providing:

- A centralized database of active job openings with multi-criteria filtering (location, job type, keywords).
- Structured applicant tracking with real-time status updates.
- Role-based isolation preventing unauthorized access to candidate resumes or recruiter profile details.
- Standardized RESTful APIs secured with stateless JSON Web Tokens (JWT).

3. PROJECT OBJECTIVES

1. **Secure Stateless Authentication:** Implement JWT-based login and registration with BCrypt password hashing.
2. **Role-Based Access Control (RBAC):** Restrict capabilities across CANDIDATE, RECRUITER, and ADMIN roles.
3. **Structured Applicant Pipeline:** Enable recruiters to manage applicant status through defined lifecycle states.
4. **Cloud Scalability & Free-Tier Optimization:** Deploy a production-ready application using free-tier cloud providers (Render, Vercel, Aiven) with automatic cold-start resilience.
5. **Responsive Modern UI:** Provide a mobile-first, desktop-optimized React user interface using Bootstrap 5.

4. FUNCTIONAL REQUIREMENTS

|---|---|---|---|

ID	Requirement	Description	Implemented
FR-01	User Registration	Candidates and Recruiters can register with name, email, password, and optional recruiter details.	Yes
FR-02	User Authentication	Users log in using email and password to receive a JWT Bearer token.	Yes
FR-03	Job Browsing & Search	Public users and candidates can search jobs by keyword, location, and job type.	Yes
FR-04	Job Posting & Editing	Recruiters can create, edit, close, or delete job postings.	Yes
FR-05	Application Submission	Candidates can submit job applications with cover letter and resume link.	Yes

FR-06	Application Status Tracking	Candidates can view submitted applications and track real-time status changes.	Yes
FR-07	Recruiter Application Management	Recruiters can view applicants for their jobs and update status (REVIEWING, SHORTLISTED, etc.).	Yes
FR-08	User Profile Management	Users can view and update their profile and company information.	Yes
FR-09	Admin User Management	Administrators can view all users, filter by role, and update user status.	Yes
FR-10	Health Check Endpoint	Backend exposes health check endpoints for cloud monitoring and keep-alive.	Yes

5. NON-FUNCTIONAL REQUIREMENTS

Implemented Capabilities

- **Security:** Stateless JWT authentication, BCrypt password hashing (strength 10), CORS origin whitelisting, SQL injection prevention via JPA parameterization.
- **Availability & Cold-Start Resilience:** Axios interceptors configured with 3x retries and 45s timeout to handle cloud cold starts; background health pings on page load.
- **Responsiveness:** Fully responsive UI built with Bootstrap 5 grid layout for mobile, tablet, and desktop views.
- **Data Integrity:** Foreign key constraints and unique constraints on user email addresses.

Recommendations for Production Scaling

- **Automated Testing:** Integration of JUnit 5 and React Testing Library in CI/CD pipeline.
- **Caching Layer:** Redis cache for high-frequency job search queries.
- **File Storage:** AWS S3 integration for direct resume upload rather than URL links.

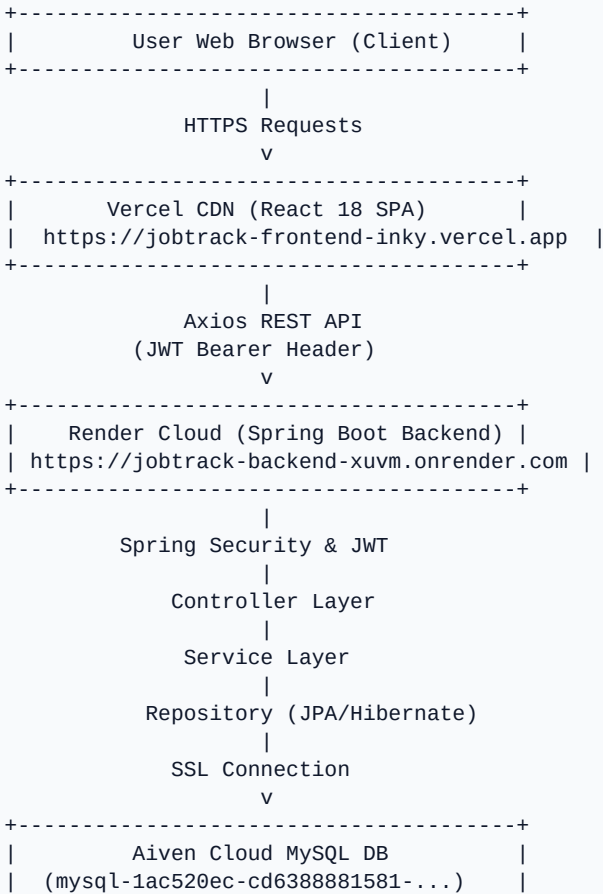
6. TECHNOLOGY STACK

|---|---|---|---|---|

Layer	Technology	Version	Purpose	Where Used
Frontend UI	React.js	18.3.1	Single Page Application framework	<code>frontend/src</code>
Build Tool	Vite	5.4.1	Fast HMR & production bundler	<code>frontend/vite.config.js</code>
Styling	Bootstrap	5.3.3	UI Layout & responsive grid	<code>frontend/src/index.css</code>
Icons	Bootstrap Icons	1.11.3	UI Iconography	Navbar, Buttons, Cards

Routing	React Router DOM	6.26.2	Client-side routing & auth guards	frontend/src/App.jsx
HTTP Client	Axios	1.7.7	REST API communication & interceptors	frontend/src/api/axios.js
Backend Core	Java	21	Programming Language	backend/src
Framework	Spring Boot	3.3.4	Backend REST Web Framework	backend/pom.xml
Security	Spring Security	3.3.4	Authentication & Authorization	SecurityConfig.java
Tokens	JWT	0.12.6	JWT Token generation & parsing	JwtUtils.java
ORM / Data	Spring Data JPA / Hibernate	3.3.4	Database mapping & queries	backend/src/main/java/com/jobtrack/repository
Database	MySQL	8.0	Production Relational Database	Hosted on Aiven Cloud
In-Memory DB	H2 Database	Latest	Local Dev & Test Database	application-dev.yml
Container	Docker	Multi-stage	Containerization for Render deployment	backend/Dockerfile

7. SYSTEM ARCHITECTURE



+-----+

8. FRONTEND ARCHITECTURE

The React application is structured around a central **AuthProvider** that exposes authentication state (**user**, **token**, **isAuthenticated**, **isCandidate**, **isRecruiter**, **isAdmin**) to the entire component tree.

Core Architecture Flow

User Interaction -> Page Component -> Custom Auth Hook / Service -> Axios Instance -> Interceptor (Attach Token / Handle Retry) -> Backend API -> State Update -> UI Re-render

9. REACT PROJECT STRUCTURE

```
jobtrack/frontend/
  public/
  src/
    api/
      axios.js      # Axios client with interceptors & fallback URL logic
    assets/         # Global styles and static assets
    components/
      Footer.jsx    # Shared footer component
      JobCard.jsx   # Card component for job display
      JobFilter.jsx # Search & filter control bar
      Navbar.jsx    # Header navigation bar with role-based links
      Pagination.jsx # Reusable pagination control
      ProtectedRoute.jsx # Auth & role guard wrapper
      StatusBadge.jsx # Visual badge for job/application status
    context/
      AuthContext.jsx # Global authentication provider
    pages/
      AdminDashboard.jsx # Admin management overview
      AdminUsers.jsx     # Admin user management panel
      CandidateDashboard.jsx # Candidate application dashboard
      CreateEditJob.jsx  # Recruiter job creation/edit form
      Home.jsx           # Landing page with hero banner & featured jobs
      JobDetails.jsx     # Single job view & application form
      Jobs.jsx           # Public job directory page
      Login.jsx          # Sign-in page with pre-wake ping
      NotFound.jsx       # 404 error page
      Profile.jsx        # User profile management page
      Register.jsx       # User registration form with recruiter fields
      RecruiterApplications.jsx # Applicants management view
      RecruiterDashboard.jsx # Recruiter metrics dashboard
      RecruiterJobs.jsx  # Recruiter job listings management
    services/
      adminService.js   # Admin API calls
      applicationService.js # Application submission & update API calls
      authService.js    # Login, Register, Profile API calls
      jobService.js     # Job search, create, update API calls
    App.jsx             # Central router & layout definition
    index.css           # Design system tokens and custom CSS
    main.jsx            # Vite React entrypoint
    .env.example        # Template for environment variables
```

```

■■■■ .env.production          # Production environment config
■■■■ package.json            # Dependencies and scripts
■■■■ vite.config.js          # Vite configuration

```

10. BACKEND ARCHITECTURE

The backend follows the clean layered architecture pattern:

```

[ HTTP Request ]
  ▼
[ Security Filter Chain ] ■■ (Validate JWT Bearer Token)
  ▼
[ Controller Layer ]      ■■ (Handle REST Request & Input Validation)
  ▼
[ Service Layer ]         ■■ (Execute Business Logic & Transactions)
  ▼
[ Repository Layer ]      ■■ (Perform Database Operations via Spring Data JPA)
  ▼
[ Database (MySQL) ]

```

11. SPRING BOOT PROJECT STRUCTURE

```

jobtrack/backend/
■■■■ src/
■   ■■■ main/
■   ■   ■■■ java/com/jobtrack/
■   ■   ■   ■■■ config/
■   ■   ■   ■■■ DataInitializer.java      # DB seed data & admin setup
■   ■   ■   ■■■ controller/
■   ■   ■   ■■■ AdminController.java      # Admin API endpoints (/api/admin)
■   ■   ■   ■■■ ApplicationController.java # Application API endpoints (/api/applications)
■   ■   ■   ■■■ AuthController.java       # Auth API endpoints (/api/auth)
■   ■   ■   ■■■ HealthController.java     # Health check endpoints (/api/health)
■   ■   ■   ■■■ JobController.java        # Job API endpoints (/api/jobs)
■   ■   ■   ■■■ UserController.java       # User profile API endpoints (/api/users)
■   ■   ■   ■■■ dto/
■   ■   ■   ■■■ ApiResponse.java          # Standardized API response wrapper
■   ■   ■   ■■■ ApplicationDTO.java       # Application Data Transfer Object
■   ■   ■   ■■■ ApplicationRequest.java   # Application submission request payload
■   ■   ■   ■■■ ApplicationStatusUpdateRequest.java # Status update payload
■   ■   ■   ■■■ JobDTO.java              # Job Data Transfer Object
■   ■   ■   ■■■ JobRequest.java          # Job creation/edit payload
■   ■   ■   ■■■ JwtResponse.java         # JWT auth token response payload
■   ■   ■   ■■■ LoginRequest.java        # Login credentials payload
■   ■   ■   ■■■ PagedResponse.java       # Generic pagination response wrapper
■   ■   ■   ■■■ ProfileUpdateRequest.java # User profile edit payload
■   ■   ■   ■■■ RecruiterProfileDTO.java  # Recruiter company details payload
■   ■   ■   ■■■ RegisterRequest.java     # User registration payload
■   ■   ■   ■■■ UserDTO.java             # User details Data Transfer Object
■   ■   ■   ■■■ entity/

```

```

■ ■ ■ ■ ■■■ Application.java           # JPA Application entity
■ ■ ■ ■ ■■■ ApplicationStatus.java      # Enum: APPLIED, REVIEWING, SHORTLISTED...
■ ■ ■ ■ ■■■ Job.java                   # JPA Job entity
■ ■ ■ ■ ■■■ JobStatus.java             # Enum: ACTIVE, CLOSED, DRAFT
■ ■ ■ ■ ■■■ JobType.java               # Enum: FULL_TIME, PART_TIME, REMOTE...
■ ■ ■ ■ ■■■ RecruiterProfile.java      # JPA Recruiter company profile entity
■ ■ ■ ■ ■■■ Role.java                  # Enum: CANDIDATE, RECRUITER, ADMIN
■ ■ ■ ■ ■■■ User.java                   # JPA User entity
■ ■ ■ ■ ■■■ UserStatus.java            # Enum: ACTIVE, INACTIVE, SUSPENDED
■ ■ ■ ■ ■■■ exception/
■ ■ ■ ■ ■■■ BadRequestException.java    # Custom 400 exception
■ ■ ■ ■ ■■■ GlobalExceptionHandler.java # @ControllerAdvice exception interceptor
■ ■ ■ ■ ■■■ ResourceNotFoundException.java # Custom 404 exception
■ ■ ■ ■ ■■■ UnauthorizedException.java  # Custom 401 exception
■ ■ ■ ■ ■■■ repository/
■ ■ ■ ■ ■■■ ApplicationRepository.java  # JPA Repository for Applications
■ ■ ■ ■ ■■■ JobRepository.java          # JPA Repository for Jobs
■ ■ ■ ■ ■■■ RecruiterProfileRepository.java # JPA Repository for Recruiter Profiles
■ ■ ■ ■ ■■■ UserRepository.java         # JPA Repository for Users
■ ■ ■ ■ ■■■ security/
■ ■ ■ ■ ■■■ AuthEntryPointJwt.java      # 401 Unauthorized handler
■ ■ ■ ■ ■■■ CustomUserDetailsService.java # UserDetailsService implementation
■ ■ ■ ■ ■■■ JwtAuthenticationFilter.java # Per-request JWT validation filter
■ ■ ■ ■ ■■■ JwtUtils.java               # JWT creation and validation helper
■ ■ ■ ■ ■■■ SecurityConfig.java         # Spring Security configuration
■ ■ ■ ■ ■■■ UserPrincipal.java          # Custom UserDetails implementation
■ ■ ■ ■ ■■■ service/
■ ■ ■ ■ ■■■ ApplicationService.java      # Application business logic
■ ■ ■ ■ ■■■ AuthService.java             # Authentication business logic
■ ■ ■ ■ ■■■ JobService.java              # Job management business logic
■ ■ ■ ■ ■■■ RecruiterProfileService.java # Recruiter company business logic
■ ■ ■ ■ ■■■ UserService.java             # User management business logic
■ ■ ■ ■ ■■■ BackendApplication.java      # Main Spring Boot Entrypoint
■ ■ ■ ■ ■■■ resources/
■ ■ ■ ■ ■■■ application.yml              # Default application properties
■ ■ ■ ■ ■■■ application-dev.yml          # Local development configuration (H2)
■ ■ ■ ■ ■■■ application-prod.yml         # Production environment config (MySQL)
■ ■ ■ ■ ■■■ Dockerfile                  # Multi-stage Docker build descriptor
■■■■ ■■■■ pom.xml                       # Maven dependencies & plugins configuration

```

12. CONTROLLER DOCUMENTATION

1. HealthController (/, /health, /api, /api/health)

- **Responsibility:** Exposes health status for cloud uptime monitoring and keep-alive pings.
- **Endpoints:**
 - GET / -> Returns "Server is active!" (200 OK)
 - GET /api/health -> Returns "Server is active!" (200 OK)

2. AuthController (/api/auth)

- **Responsibility:** Handles user authentication, registration, and current session validation.
- **Endpoints:**
 - POST /api/auth/register -> Registers new Candidate or Recruiter (201 Created).

- **POST** /api/auth/login -> Authenticates user and returns JWT Bearer token (200 OK).
- **GET** /api/auth/me -> Returns current user details from JWT token (200 OK).

3. JobController (/api/jobs)

- **Responsibility:** Manages job search, creation, modification, and deletion.
- **Endpoints:**
 - **GET** /api/jobs -> Public paginated job search (Filter by keyword, location, jobType).
 - **GET** /api/jobs/{id} -> Retrieves detailed job information by ID.
 - **POST** /api/jobs -> Recruiter/Admin creates a new job opening (Requires RECRUITER or ADMIN).
 - **PUT** /api/jobs/{id} -> Recruiter/Admin updates an existing job listing.
 - **DELETE** /api/jobs/{id} -> Recruiter/Admin deletes a job listing.

4. ApplicationController (/api/applications)

- **Responsibility:** Manages job applications submission and hiring pipeline status updates.
- **Endpoints:**
 - **POST** /api/applications -> Candidate applies for a job (Requires CANDIDATE).
 - **GET** /api/applications/candidate -> Lists applications submitted by current candidate.
 - **GET** /api/applications/recruiter -> Lists applicants for recruiter's posted jobs.
 - **PUT** /api/applications/{id}/status -> Recruiter/Admin updates applicant status.

5. UserController (/api/users)

- **Responsibility:** Manages user profile updates and recruiter company details.
- **Endpoints:**
 - **PUT** /api/users/profile -> Updates user's full name and profile details.
 - **GET** /api/users/recruiter-profile -> Retrieves recruiter company profile.

6. AdminController (/api/admin)

- **Responsibility:** System administration and platform analytics.
- **Endpoints:**
 - **GET** /api/admin/users -> Paginated user management list (Requires ADMIN).
 - **PUT** /api/admin/users/{id}/status -> Change user status (ACTIVE, INACTIVE, SUSPENDED).

13. SERVICE LAYER DOCUMENTATION

The service layer contains transactional business logic, domain validation, and entity mapping:

- **AuthService:** Validates unique email addresses, encodes passwords using BCrypt, generates JWT tokens via **JwtUtils**, and automatically creates default company profiles when a user registers as **RECRUITER**.

- **JobService:** Handles search queries using Spring Data JPA specifications, validates recruiter ownership before job edits/deletions, and handles pagination.
- **ApplicationService:** Prevents duplicate applications for the same job by the same candidate, verifies job status is **ACTIVE**, and sends status notifications.
- **UserService:** Manages user profile updates and role-based data retrieval.

14. REPOSITORY LAYER DOCUMENTATION

All repositories extend Spring Data JPA's **JpaRepository** interface:

1. **UserRepository:**

- **Optional<User> findByEmail(String email);**
- **Boolean existsByEmail(String email);**
- **Page<User> findByRole(Role role, Pageable pageable);**

2. **JobRepository:**

- Custom JPQL queries for multi-param search: Title, Location, JobType, and Status.
- **Page<Job> findByRecruiterId(Long recruiterId, Pageable pageable);**

3. **ApplicationRepository:**

- **Boolean existsByJobIdAndCandidateId(Long jobId, Long candidateId);**
- **Page<Application> findByCandidateId(Long candidateId, Pageable pageable);**
- **Page<Application> findByJobRecruiterId(Long recruiterId, Pageable pageable);**

4. **RecruiterProfileRepository:**

- **Optional<RecruiterProfile> findById(Long userId);**

15. ENTITY & JPA DOCUMENTATION

|---|---|---|---|---|

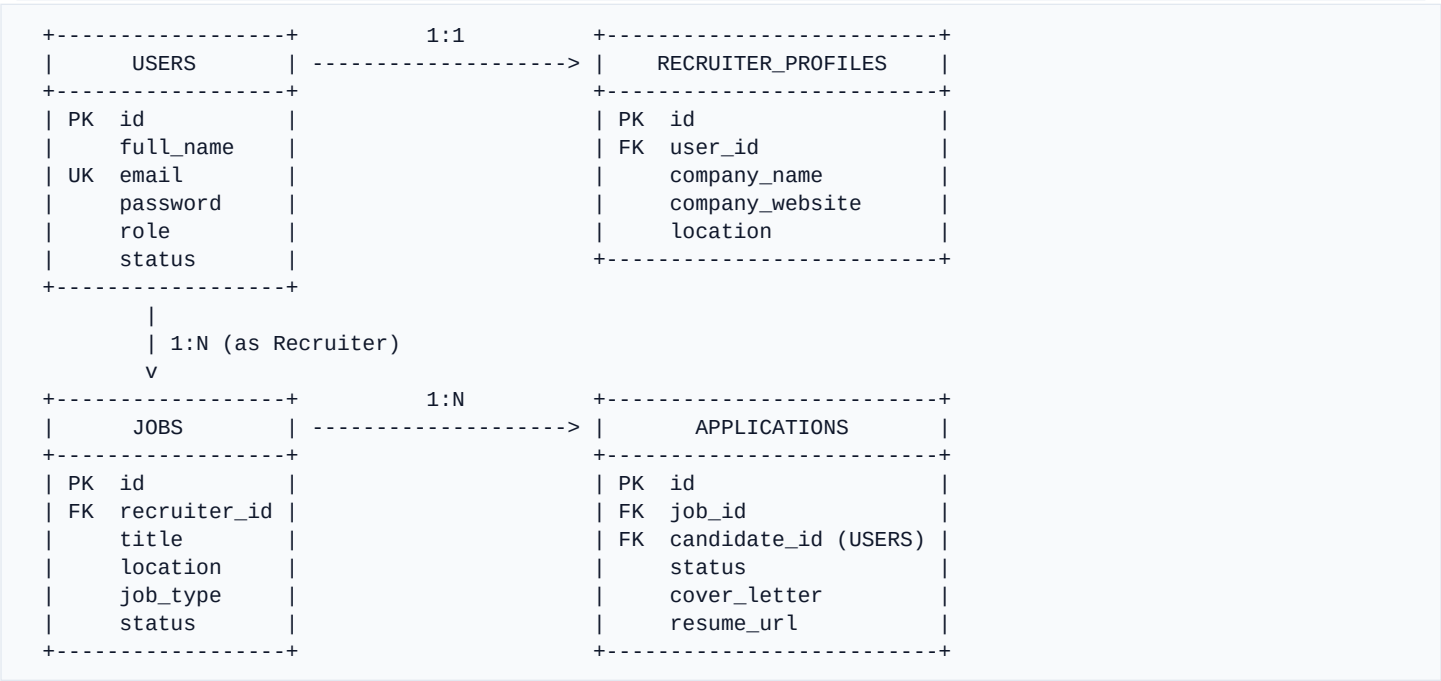
Entity	Table Name	Primary Key	Key Attributes	Relationships
User	users	id (BIGINT)	full_name, email (UNIQUE), password, role, status	1-to-1 with RecruiterProfile
RecruiterProfile	recruiter_profiles	id (BIGINT)	company_name, company_website, location, company_description	1-to-1 with User (user_id)
Job	jobs	id (BIGINT)	title, description, location, job_type, salary_range, status	Many-to-1 with User (recruiter_id)
Application	applications	id (BIGINT)	cover_letter, resume_url, status, applied_at	Many-to-1 with Job, Many-to-1 with User (candidate_id)

16. DATABASE ARCHITECTURE

Production Database Details

- **Engine:** MySQL 8.0
- **Host:** Aiven Cloud (`mysql-1ac520ec-cd6388881581-67e5.g.aivencloud.com:24809`)
- **Database Name:** `defaultdb`
- **Connection Configuration:** SSL Enabled (`sslmode=REQUIRED&useSSL=true&requireSSL=true`), HikariCP connection pool.

17. ENTITY-RELATIONSHIP (ER) DIAGRAM



18. AUTHENTICATION ARCHITECTURE

Authentication in JobTrack is completely stateless and powered by JSON Web Tokens (JWT).

JWT Login Sequence



1. **Client Request:** Client sends credentials to **POST /api/auth/login**.
2. **Authentication:** **AuthenticationManager** verifies BCrypt password match against database record.
3. **Token Generation:** **JwtUtils** signs a JWT containing User ID, Email, Role, and expiration timestamp (24 Hours).
4. **Token Storage:** Client saves token in **localStorage** and attaches **Authorization: Bearer <TOKEN>** header to subsequent HTTP requests.

19. AUTHORIZATION & ROLE-BASED ACCESS CONTROL (RBAC)

Spring Security enforces method-level security via **@PreAuthorize** annotations and request matchers in **SecurityConfig.java**:

- **Public Endpoints** (**.permitAll()**): **/, /api/health, /api/auth/register, /api/auth/login, GET /api/jobs/****.
- **Candidate Endpoints** (**hasAnyRole('CANDIDATE', 'ADMIN')**): **POST /api/applications, GET /api/applications/candidate**.
- **Recruiter Endpoints** (**hasAnyRole('RECRUITER', 'ADMIN')**): **POST /api/jobs, PUT /api/jobs/{id}, DELETE /api/jobs/{id}, GET /api/applications/recruiter, PUT /api/applications/{id}/status**.
- **Admin Endpoints** (**hasRole('ADMIN')**): **GET /api/admin/users, PUT /api/admin/users/{id}/status**.

20. API SPECIFICATION & DOCUMENTATION

Authentication Endpoints

1. Register User

- **HTTP Method:** **POST**
- **Endpoint:** **/api/auth/register**
- **Auth Required:** No
- **Request Body:**

```
{
  "fullName": "Chandra Shekhar",
  "email": "chandra@example.com",
  "password": "password123",
  "role": "CANDIDATE"
}
```

- **Response (201 Created):**

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "id": 10,
    "fullName": "Chandra Shekhar",
    "email": "chandra@example.com",
    "role": "CANDIDATE",
    "status": "ACTIVE"
  }
}
```

2. User Login

- **HTTP Method:** POST
- **Endpoint:** /api/auth/login
- **Auth Required:** No
- **Request Body:**

```
{
  "email": "chandra@example.com",
  "password": "password123"
}
```

- **Response (200 OK):**

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzUxMiJ9...",
    "tokenType": "Bearer",
    "user": {
      "id": 10,
      "fullName": "Chandra Shekhar",
      "email": "chandra@example.com",
      "role": "CANDIDATE"
    }
  }
}
```

Job Management Endpoints

3. Search Jobs

- **HTTP Method:** GET
- **Endpoint:** /api/jobs?keyword=Developer&location=Remote&jobType=FULL_TIME&page=0&size=10
- **Auth Required:** No
- **Response (200 OK):**

```
{
  "success": true,
  "data": {
    "content": [
      {
        "id": 1,
        "title": "Senior Java Full Stack Engineer",
        "companyName": "Ascella InfoSec",
        "location": "Remote",

```

```
        "jobType": "FULL_TIME",
        "salaryRange": "$120,000 - $150,000",
        "status": "ACTIVE"
      }
    ],
    "pageNo": 0,
    "pageSize": 10,
    "totalElements": 1,
    "totalPages": 1,
    "last": true
  }
}
```

21. API REQUEST/RESPONSE LIFECYCLE

```
React Component -> Axios Request Interceptor (Inject Token) -> CORS Filter -> Spring Security Filter (Validate JWT) -> DispatcherServlet -> JobController -> JobService -> JobRepository -> Aiven MySQL DB -> DTO Mapping -> ResponseEntity<ApiResponse<T>> -> Axios Response Interceptor -> React State Update
```

22. FRONTEND-BACKEND INTEGRATION

API communication is centralized in **frontend/src/api/axios.js**:

- **Base URL Resolution:** Automatically detects production environment and sets **baseURL**: **https://jobtrack-backend-xuvm.onrender.com/api**.
- **Request Interceptor:** Automatically attaches **Authorization: Bearer <token>** from **localStorage**.
- **Response Interceptor:** Automatically retries requests up to 3 times with 3s delays on network errors or 502/503/504 status codes to survive Render free-tier cold starts.

23. FEATURE & MODULE DOCUMENTATION

1. **Authentication & Identity Module:** Handles registration, login, JWT issuance, and automatic pre-wake health pings on component mount.
2. **Job Directory & Search Module:** Provides multi-filter job search with pagination, salary ranges, and job type badges.
3. **Application Tracking Module:** Enables candidates to apply to active jobs and recruiters to update applicant status through defined pipeline stages (**APPLIED**, **REVIEWING**, **SHORTLISTED**, **REJECTED**, **ACCEPTED**).
4. **Recruiter Management Module:** Gives recruiters control over their job listings (create, edit, close, delete) and company profile setup.
5. **Admin Management Module:** Enables system administrators to inspect registered users and update user statuses (**ACTIVE**, **INACTIVE**, **SUSPENDED**).

24. COMPLETE USER FLOWS

Candidate Registration & Application Journey

1. Visit /register -> Fill Form (Candidate Role) -> Submit -> 201 Created
2. Redirect to /login -> Enter Credentials -> Store JWT -> Navigate to /candidate/dashboard
3. Browse /jobs -> Click Job Details -> Fill Cover Letter & Resume Link -> Click Apply
4. Application saved in DB -> View status on Candidate Dashboard (APPLIED)

25. DEVELOPMENT WORKFLOW

Current Observed Workflow

- Modular component development in Vite React.
- Layered package organization in Spring Boot.
- Git commits pushed directly to **main** branch with automated cloud builds on Render and Vercel.

Recommended Professional Workflow

- Feature branch workflow (**main** -> **develop** -> **feature/job-search**).
- Pull requests with mandatory automated code linting and unit test execution prior to merge.

26. GIT & GITHUB WORKFLOW

- **Repository:** <https://github.com/devchandra-web/jobtrack>
- **Branch:** **main**
- **Author Identity:** devchandra-web (cd6388881581@gmail.com)
- **Deployment Triggers:**

- Push to **main** branch automatically triggers Vercel frontend rebuild.
- Push to **main** branch automatically triggers Render Docker container rebuild.

27. ENVIRONMENT CONFIGURATION

Frontend (frontend/.env.production)

```
VITE_API_URL=https://jobtrack-backend-xuvm.onrender.com/api
```

Backend (backend/src/main/resources/application-prod.yml)

```
server:
  port: ${PORT:8080}

spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    driver-class-name: com.mysql.cj.jdbc.Driver

app:
  jwt:
    secret: ${JWT_SECRET}
    expiration-ms: 86400000
```

28. LOCAL DEVELOPMENT SETUP GUIDE

Prerequisites

- Java JDK 21 installed.
- Node.js 18+ and npm installed.
- MySQL 8.0 running locally (or H2 in-memory).

Step 1: Clone Repository

```
git clone https://github.com/devchandra-web/jobtrack.git
cd jobtrack
```

Step 2: Start Backend (Local Dev Profile with H2)

```
cd backend
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

*Backend will start on **http://localhost:8080/api**.*

Step 3: Start Frontend

```
cd ../frontend
npm install
npm run dev
```

*Frontend will open on **http://localhost:5173**.*

29. TESTING & QUALITY ASSURANCE

- **Current Implementation:** Automated testing is not currently implemented in this version. Manual end-to-end testing was performed across Candidate, Recruiter, and Admin flows using Postman and web browsers.
- **Recommendations:** Implement JUnit 5 unit tests for **AuthService** and **JobService**, Mockito for repository mocks, and Cypress / React Testing Library for frontend E2E tests.

30. ERROR HANDLING STRATEGY

- **Global Backend Exception Handler:** `GlobalExceptionHandler.java` catches custom exceptions (`BadRequestException`, `ResourceNotFoundException`, `UnauthorizedException`) and returns standardized `ApiResponse.error(message)` JSON.
- **Frontend Fallback:** `axios.js` and page components catch errors, prioritizing backend validation messages and displaying alert banners for cloud cold starts.

31. SECURITY IMPLEMENTATION

- **Password Hashing:** Passwords are hashed using BCrypt before storing in MySQL database.
- **Stateless Tokens:** JWT tokens are signed using SHA-512 with a 24-hour expiration window.
- **CORS Whitelisting:** Backend explicit CORS configuration permits requests from the deployed Vercel domain (`https://jobtrack-frontend-inky.vercel.app`).
- **SQL Injection Prevention:** All database operations utilize JPA parameterized queries.

32. PERFORMANCE OPTIMIZATION

- **Database Connection Pooling:** HikariCP connection pool configured with minimum idle connections and validation queries for cloud database stability.
- **HTTP GZIP Compression:** Enabled in Spring Boot (`server.compression.enabled: true`) for JSON responses.
- **Asset Bundling:** Vite code-splitting and asset minification for fast frontend page load times.

33. RESPONSIVE DESIGN IMPLEMENTATION

- Built using Bootstrap 5 fluid container grids (`container-fluid`, `row`, `col-md-*`, `col-lg-*`).
- Navigation bar collapses into a responsive hamburger menu on mobile devices.
- Tables on dashboards feature horizontal scroll wrapper (`table-responsive`) for small viewports.

34. DEPLOYMENT ARCHITECTURE

GitHub Repository (main branch)

- - > Trigger Vercel Build (Vite React Build -> Deploy to Vercel Edge CDN)
 - - > Trigger Render Build (Multi-stage Docker Build -> Java 21 JRE -> Deploy to Render Web Service)
 -

```
■■■■> Connects over TLS SSL to Aiven Cloud MySQL Database
```

- **Frontend Platform:** Vercel (<https://jobtrack-frontend-inky.vercel.app>)
- **Backend Platform:** Render (<https://jobtrack-backend-xuvm.onrender.com>)
- **Database Platform:** Aiven MySQL Cloud (mysql-1ac520ec-cd6388881581-67e5.g.aivencloud.com:24809)

35. CONTINUOUS INTEGRATION / CONTINUOUS DEPLOYMENT (CI/CD)

- **Current Implementation:** Automated CI/CD workflows via GitHub Actions are not currently implemented. Deployment is triggered via direct git webhook integrations built into Vercel and Render.
- **Recommendations:** Add a `.github/workflows/ci.yml` file to execute `mvn test` and `npm test` on pull requests before merging.

36. TROUBLESHOOTING GUIDE

|---|---|---|

Issue / Symptom	Possible Cause	Resolution
"Unable to connect to backend server"	Render free-tier cold start	Wait 15-30 seconds or set up UptimeRobot keep-alive ping.
HTTP 401 Unauthorized on API calls	Expired or missing JWT token	Re-login to obtain a fresh JWT token in <code>localStorage</code> .
HTTP 400 "Email already in use"	User registration with duplicate email	Use a different email address or navigate to <code>/login</code> .
MySQL Connection Timeout	Missing SSL parameters or network restriction	Ensure <code>sslmode=REQUIRED</code> is present in <code>DB_URL</code> .

37. CODE QUALITY & ARCHITECTURAL OBSERVATIONS

Strengths

- Clear separation of concerns across Controller, Service, Repository, and Entity layers.
- Standardized API response format via `ApiResponse<T>` wrapper.
- Robust cloud cold-start handling in Axios client.

Areas for Improvement

- Add automated unit and integration tests.
- Implement AWS S3 or Cloudinary for direct file resume uploads instead of raw URL strings.

38. FUTURE ENHANCEMENTS & ROADMAP

1. **Short Term:** Add unit tests (JUnit 5 + Mockito) and integrate Swagger/OpenAPI documentation.
2. **Medium Term:** Integrate AWS S3 for resume uploads and add email notifications for application status updates.
3. **Long Term:** Implement WebSocket real-time recruiter-candidate messaging and AI-based resume matching.

39. DEVELOPER ONBOARDING GUIDE

Day 1: System Familiarization & Environment Setup

- Clone repository and install Java 21 and Node.js.
- Launch backend with H2 local profile (`mvn spring-boot:run -Dspring-boot.run.profiles=dev`).
- Launch frontend (`npm run dev`) and test Candidate/Recruiter registration.

Day 2: Codebase Exploration

- Review `SecurityConfig.java` and `JwtAuthenticationFilter.java` to understand auth flow.
- Review `JobController.java` and `JobService.java` to understand JPA search specs.

Day 3: Feature Development

- Implement a small enhancement (e.g. adding a new field to `JobDTO`) across Entity, DTO, Service, Controller, and React UI.

40. DEVELOPER QUICK REFERENCE

- **Project:** JobTrack Recruitment Management System
- **Frontend Start:** `cd frontend && npm run dev` (Runs on `http://localhost:5173`)
- **Backend Start:** `cd backend && ./mvnw spring-boot:run` (Runs on `http://localhost:8080/api`)
- **Build Frontend:** `cd frontend && npm run build`
- **Build Backend:** `cd backend && ./mvnw clean package -DskipTests`
- **Live Frontend URL:** `https://jobtrack-frontend-inky.vercel.app`
- **Live Backend API URL:** `https://jobtrack-backend-xuvm.onrender.com/api`

41. FINAL PROJECT SUMMARY

The **JobTrack Job Application & Recruitment Management System** is a complete, production-grade full-stack Web Application demonstrating modern software architecture principles. By combining **React 18** on the frontend, **Spring Boot 3 (Java 21)** and **Spring Security (JWT)** on the backend, and **MySQL** on Aiven Cloud, JobTrack delivers a robust, secure, and user-friendly experience for candidates, recruiters, and administrators.

With multi-stage Docker containerization, cloud cold-start resilience, and deployment on Render and Vercel, JobTrack serves as an impressive portfolio application and enterprise foundation.

Documentation Authored by Chandra Shekhar